Optimising Weak Reference Processing in OpenJDK's ZGC

Master's Thesis in Computer Science

Fredrik Hammarberg

MSc Computer and Information Engineering, Uppsala University

fredrik.hammarberg00@outlook.com

May 2026

Master's Thesis Defence – Uppsala University

UPPSALA
UNIVERSITET

Agenda

Introduction & Motivation

GC Pauses Are a Real Cost

- Java manages memory automatically: you allocate with `new`, the garbage collector reclaims unreachable objects
- GC pauses disrupt latency-sensitive workloads – microservices, trading systems, real-time analytics
- **ZGC**: achieves sub-millisecond stop-the-world pauses regardless of heap size, by doing most GC work concurrently

One bottleneck remains

Weak reference processing cost scales linearly with the number of `WeakReference` objects. Large applications hold *tens of millions* – yet ZGC applies the same costly pipeline to all of them, even when most do not need it.

Why WeakReferences Are Everywhere

- A `WeakReference<T>` lets you hold a reference to an object that the GC is *still allowed* to collect
- Used in: `WeakHashMap`, Guava Cache, Caffeine, many ORM frameworks, canonicalisation tables
- Optional `ReferenceQueue`: GC notifies you when the referent is collected
- **Most** `WeakReference`s are created *without* a queue – they only need weak reachability, not the callback

Common (no queue)

```
new WeakReference<>(obj)
```

Pure weak reachability.
No notification needed.

Less common (with queue)

```
new WeakReference<>(obj, q)
```

GC notifies via `q` on collection.

Research Questions

RQ1 – Pipeline Optimisations

How do targeted modifications to ZGC's processing of `WeakReference` objects *without* a `ReferenceQueue` affect reference-processing time, total GC cycle time, GC memory, and Java heap usage?

RQ2 – Weak Fields

How does expressing weak semantics through *annotated object fields* affect the same metrics compared to the optimised `WeakReference` pipeline approach?

Background

Java Memory Management and GC

- All Java objects live on the **heap**; you allocate with `new` and the GC reclaims unreachable objects
- An object is *live* if reachable from a **GC root**: thread stacks, static fields, JNI references
- GC cycle: **mark** live objects → **process references** → **relocate**
- **Generational**: young gen (minor cycles, frequent) + old gen (**major cycles**, infrequent)

Object liveness

GC Root
- Object A — live
  - Object B — live

(unreachable)
- Object C — collected

Java Reference Types

- **Strong** (`Object o = new Foo()`)
  object lives as long as `o` is in scope
- **Weak** (`new WeakReference<>(obj)`)
  GC *may* collect the referent; `.get()` returns `null` afterwards
- **Soft**: like weak but GC uses a last-resort heuristic (memory pressure)
- **Phantom**: always returns `null`; used for post-mortem resource cleanup

ReferenceQueue

An optional notification channel: the GC deposits cleared `Reference` objects here.

Useful for explicit cleanup – but the *majority* of `WeakReference`s are queue-less.

ZGC: Concurrent, Sub-Millisecond GC

- Most marking and relocation happens **concurrently** with the application – stop-the-world pauses stay < 1 ms
- **Coloured pointers**: metadata bits embedded in the 64-bit pointer itself (mark state, relocation status)
- **Load barriers**: small code sequences injected at pointer loads; let the GC update object state on-the-fly
- Handles heaps from 8 GB to 16 TB with the same pause budget

Generational ZGC (JDK 21+)

Young generation collected by minor cycles; old generation by less frequent **major** cycles. Reference processing runs during the major-cycle marking pause.

Weak References in ZGC's Pipeline

1. **Discovery** – while marking, reference objects are appended to per-worker *intrusive linked lists* via a hidden `discovered` field
2. **Processing** – check each referent's liveness; if unreachable, atomically clear the `referent` field (CAS + load barrier)
3. **Enqueue** – cleared references added to the *pending list*; the `ReferenceHandler` thread delivers them to their `ReferenceQueue`

The cost of uniformity

Steps 1–3 apply to *all* weak references – including queue-less ones. A queue-less reference traverses the full pipeline only to be silently dropped at step 3, wasting linked-list traversal, CAS operations, and `ReferenceHandler` wake-ups.

Design & Implementation

Four Mechanisms, Eight Variants

Three **pipeline optimisations** targeting queue-less refs:

- **sep** – split the discovered list at discovery time
- **dyn** – replace the linked list with a dynamic array
- **clear_path** – specialised clear (no CAS)

One **structural alternative**:

- **weak_fields** – `@weak` field annotation

All $2^3 = 8$ pipeline combinations benchmarked independently.

Variant	cp	sep	dyn
`none`	–	–	–
`dyn`	–	–	✓
`sep`	–	✓	–
`sep_dyn`	–	✓	✓
`cp`	✓	–	–
`cp_dyn`	✓	–	✓
`cp_sep`	✓	✓	–
`all`	✓	✓	✓

Mechanism 1: Skip-Enqueue Separation (sep)

Idea: At discovery time, route queue-less refs to a *separate list* that never enters the pending list or `ReferenceHandler`.

Listing 1: Discovery split – pseudocode

```
bool discover_reference(oop ref, ReferenceType type) {
    if (!should_discover(ref, type)) return false,
    if (type == REF_WEAK && !has_reference_queue(ref)) {
        // Queue-less: separate list, never enqueued
        no_queue_list_per_worker.append(ref),
    } else {
        // Queue-backed and all other types: standard path
        discovered_list_per_worker.append(ref),
    }
    mark_discovered(ref),  return true,
}
```

- Per-worker lists – no inter-thread contention during discovery
- Separate processing loop that clears but never enqueues

Mechanism 2: Dynamic Array (dyn)

Idea: Replace the intrusive linked list with a contiguous array for cache-friendly traversal and pre-loaded referent data.

Listing 2: Array-based processing – pseudocode

```
void process_no_queue_refs(ZWeakRefArray& arr) {
    size_t kept = 0,
    for (size_t i = 0, i < arr.length(), i++) {
        const ZWeakRefData& d = arr.at(i),  // sequential: cache-friendly
        d.mark_not_discovered(),
        if (try_make_inactive(d)) { cleared++, }
        else { kept++, }
    }
    arr.clear_and_reserve(kept),  // retain capacity for next cycle
}
```

- Array grows on the C heap; amortised O(1) append during discovery
- Each entry *pre-loads* the referent address and value

Mechanism 3: Specialised Clear Path (clear_path)

Idea: For old-generation queue-less refs, replace the CAS-based clear with a plain store.

Standard clear

1. Load barrier on referent
2. CAS: referent $\rightarrow$ coloured null

Specialised clear

1. Use pre-loaded value (no barrier)
2. Direct store (no CAS)

Why safe?

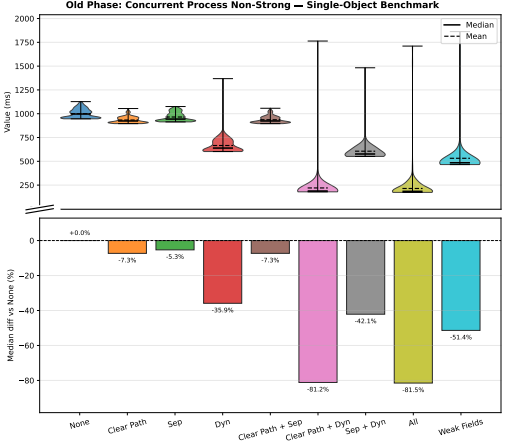
During the relevant STW pause, no application thread can update the referent of an old-generation queue-less reference – only the GC can. The CAS is therefore unnecessary.

Superadditive Interaction: dyn + clear_path

- **dyn alone**: -36% (cache locality, but still CAS)
- **clear_path alone**: -7% (faster clear, still linked list)
- **dyn + clear_path**: $-\mathbf{81}\%$

Why superadditive?

`dyn` pre-loads referent data during discovery. `clear_path` *uses* that data – eliminating the load barrier *and* the pointer-chase simultaneously.



Non-strong processing time (single-object benchmark)

Mechanism 4: Weak Fields (`@weak`)

Idea: Weakness as a *field property* – no `WeakReference` wrapper object needed.

Usage

```
import java.lang.ref.weak;  class Holder { @weak Target ref; }
```

- Annotation recognised by the class-file parser (ZGC-only; changes across 27 files in compiler, interpreter, JIT, GC)
- Fields discovered during old-generation marking; stored in per-worker `ZWeakFieldArray`
- Unreachable referent: field zeroed directly – no pending list, no `ReferenceHandler`
- **Key:** no wrapper objects allocated or promoted to old gen

Results & Analysis

Benchmark Setup

Single-object

- 20 M holder objects → one shared target
- Target released; `System.gc()` triggered
- Stress-tests reference-processing in isolation

Multi-object

- 2 M objects, random payloads, 5 GC rounds
- 20% of strong refs released per round
- More realistic liveness and locality

Environment

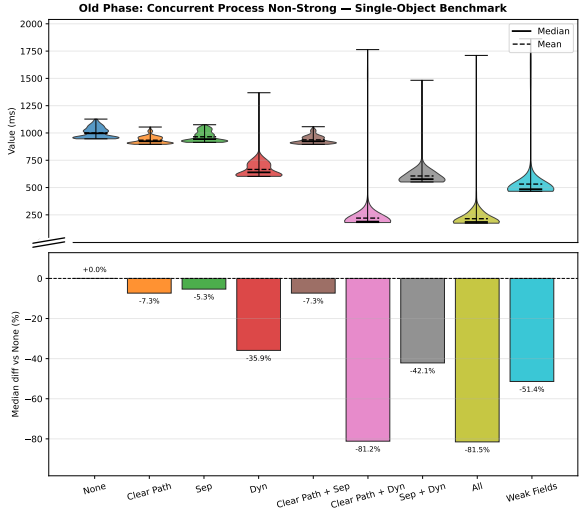
UPPMAX supercomputer (SLURM)
48-core nodes, 100 GB heap
`-XX:+UseZGC`, 250 iterations
4 instances pinned to isolated CPU sets

Metric

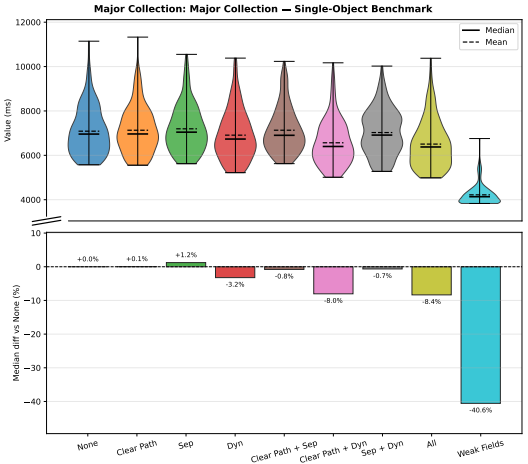
Major GC cycle time and per-phase timings via `-Xlog:gc+stats`. Reported as **median** over 250 iterations.

Non-Strong Processing Time

Variant	Median (ms)	
none	996.9	
sep	943.8	(−5%)
cp	923.7	(−7%)
dyn	639.1	(−36%)
sep_dyn	576.9	(−42%)
cp_dyn	187.9	(−**81%**)
all	184.4	(−**81%**)
wf	484.6	(−51%)



Major Collection Time

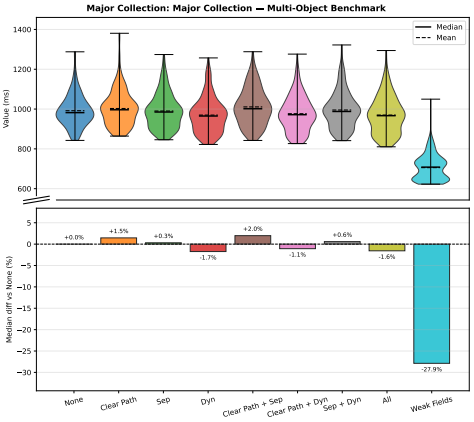


Variant	Median (ms)
`none`	6,958
`all`	6,377 (−8%)
`wf`	4,136 (−**41%**)

The ceiling

Non-strong = **14.3%** of total major GC time. Even an 81% reduction there moves the total by at most ≈11%.

Multi-Object Benchmark and Memory



Major GC time, multi-object benchmark

- Non-strong = only **4.5%** here
- Pipeline variants ≈ baseline

Auxiliary GC memory (single-object):

Variant	MB	
`none`	120	
`dyn`	308	(×2.6)
`cp_dyn`	1,268	(×**10.6**)
`all`	884	(×7.4)
`wf`	446	(×3.7)

Java heap

`wf`: 806 MB vs 1,720 MB (−53%) – no wrappers

Structural vs. Algorithmic Advantage

Pipeline optimisations

- Accelerate processing of existing `WeakReference` objects
- Gains *confined* to the non-strong processing phase

Weak fields

- Eliminate wrapper objects entirely
- Fewer objects ⇒ less work in *every* GC phase:
  marking (−31%), young-gen (−44%)

Single-object benchmark

Concurrent mark:
`none`: 2,575 ms
`wf`: 1,782 ms (−31%)

Young-gen (Promote All):
`none`: 3,201 ms
`wf`: 1,784 ms (−44%)

Conclusions

Limitations

- **Synthetic benchmarks**: single-object maximises ref-processing load; real applications have varied reference patterns and lifetimes
- **ZGC-specific**: all optimisations are ZGC-only; not applicable to G1GC or other collectors
- **One hardware platform**: UPPMAX supercomputer – results may differ on desktop, server, or cloud hardware
- **`@weak` is a prototype**: changes span 27 files across compiler, interpreter, JIT, and GC – not production-ready

Conclusions

RQ1 – Pipeline Optimisations

- `cp_dyn` and `all`: **81%** reduction in non-strong processing time (superadditive interaction)
- Major collection improvement: at most **8%** – bounded by non-strong processing being only 14.3% of total

RQ2 – Weak Fields

- **41%** major GC time reduction; **53%** heap reduction
- Structural advantage: eliminating wrapper objects benefits every GC phase

Overall Finding

Weak-reference overhead is a **representation problem**, not a pipeline problem. Meaningful improvement requires rethinking how weak semantics is encoded in the language.

Thank You

Questions?

fredrik.hammarberg00@outlook.com

References I

Back to start

[1] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” https://openjdk.org/jeps/333, 2018.
Accessed: 2026-03-12.

[2] OpenJDK, “Jep 439: Generational zgc.” https://openjdk.org/jeps/439, 2023.
Accessed: 2026-03-12.

[3] E. Österlund, “JVMLS – generational ZGC and beyond.” https://inside.java/2023/08/31/generational-zgc-and-beyond/, 2023.
Accessed: 2026-05-25.

References II

[4] Oracle, “java.lang.ref (java se 25).” https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html, 2025. Accessed: 2026-03-12.

[5] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.” https://bugs.openjdk.org/browse/JDK-8029205, n.d. Accessed: 2026-02-24.

[6] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007. Accessed: 2026-04-07.

References III

[7] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*.
Chapman and Hall/CRC, 2011.

[8] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981.
Accessed: 2026-02-24.